

Sprawozdanie z projektu

System detekcji anomalii kart płatniczych
z wykorzystaniem Apache Kafka, Apache Flink i MongoDB

Autorzy: Cyprian Szczepański i Filip Zependowski

Spis treści

1. Cel i zakres projektu.....	3
2. Potrzeby klienta i wymagania.....	3
3. Architektura systemu.....	3
4. Konfiguracja środowiska.....	5
5. Struktura projektu.....	6
6. Implementacja detektora Flink.....	6
7. Reguły detekcji anomalii.....	8
8. Alarm-watcher i zapis do MongoDB.....	9
9. Uruchamianie i testowanie systemu.....	11
10. Wyniki działania.....	12
11. Wnioski.....	13

1. Cel i zakres projektu

System przyjmuje jako dane wejściowe transakcje kart płatniczych zapisane w formacie JSON i publikowane do topicu Kafka transactions. Każda transakcja zawiera m.in. identyfikator karty, użytkownika, czas wykonania operacji, lokalizację, kwotę, pozostały limit oraz nazwę sprzedawcy.

Przetwarzanie danych odbywa się w aplikacji Apache Flink, która działa jako detektor anomalii. Aplikacja analizuje kolejne transakcje w czasie zbliżonym do rzeczywistego, porównuje je z historią danej karty i sprawdza zestaw reguł wykrywających nietypowe zachowania.

Wynikiem działania systemu są alerty publikowane do topicu Kafka alerts. Alert zawiera informacje o wykrytym typie anomalii, poziomie istotności, opisie zdarzenia oraz pełny zapis transakcji, która spowodowała jego wygenerowanie. Następnie alerty są odczytywane przez komponent alarm-watcher, prezentowane w terminalowym dashboardzie i zapisywane w bazie MongoDB.

2. Potrzeby klienta i wymagania

Potrzeby klienta

Klientem docelowym systemu jest instytucja finansowa (bank lub operator kart płatniczych), która potrzebuje narzędzia do bieżącego monitorowania transakcji kartowych pod kątem nadużyć. Z analizy potrzeb klienta wynikają następujące oczekiwania:

Wykrywanie nadużyć w czasie zbliżonym do rzeczywistego — opóźnienie między wykonaniem podejrzanej transakcji a wygenerowaniem alertu musi być na tyle małe, aby analityk mógł zareagować, zanim dojdzie do kolejnych prób oszustwa (np. zablokować kartę).

Wykrywanie wielu typów anomalii jednocześnie — pojedyncza reguła nie wystarcza; klient potrzebuje zestawu niezależnych reguł (nietypowo duże kwoty, niemożliwa podróż, seria transakcji w krótkim czasie, wyczerpywanie limitu, kwoty tuż poniżej progów kontrolnych).

Czytelna prezentacja alertów dla operatora — analityk powinien widzieć napływające alerty na żywo, móc je filtrować i przeglądać szczegóły pojedynczego zdarzenia bez przerywania pracy systemu.

Trwały zapis historii alertów — alerty muszą być przechowywane w bazie danych w celu późniejszej analizy, raportowania i jako materiał dowodowy.

Skalowalność i niezależność komponentów — wzrost liczby transakcji nie może wymagać przebudowy systemu; awaria lub restart jednego komponentu (np. dashboardu) nie może zatrzymywać detekcji ani powodować utraty danych.

Łatwe uruchomienie środowiska — całość infrastruktury powinna dać się postawić jednym poleceniem, bez ręcznej instalacji poszczególnych usług.

Wymagania funkcjonalne

ID	Wymaganie
WF-01	System przyjmuje transakcje kart płatniczych w formacie JSON publikowane do topicu Kafka transactions. Każda transakcja zawiera m.in. identyfikator karty, użytkownika, czas, lokalizację GPS, kwotę, pozostały limit i nazwę sprzedawcy.

WF-02	System analizuje każdą transakcję w aplikacji Apache Flink i porównuje ją z historią danej karty (stan per karta).
WF-03	System wykrywa anomalię typu large_amount — kwotę znacząco odbiegającą od typowych kwot danej karty (średnia, odchylenie standardowe, z-score).
WF-04	System wykrywa anomalię typu high_frequency — dużą liczbę transakcji jednej karty w krótkim oknie czasowym.
WF-05	System wykrywa anomalię typu limit_exhaustion — pojedynczą transakcję zużywającą co najmniej 95% dostępnego limitu karty.
WF-06	System wykrywa anomalię typu structuring — kwoty tuż poniżej progów kontrolnych (np. 500, 1000, 5000 PLN).
WF-07	System wykrywa anomalię typu impossible_travel — użycie tej samej karty w dwóch odległych lokalizacjach w czasie wymagającym nierealnej prędkości podróży.
WF-08	Jedna transakcja może wygenerować wiele alertów, jeżeli spełnia warunki kilku reguł jednocześnie.
WF-09	Alert publikowany do topicu Kafka alerts zawiera typ anomalii, poziom istotności (severity), opis zdarzenia oraz pełny zapis transakcji źródłowej.
WF-10	Komponent alarm-watcher konsumuje alerty z topicu alerts i prezentuje je na żywo w terminalowym dashboardzie.
WF-11	Alarm-watcher umożliwia filtrowanie alertów: tekstowo (po identyfikatorze karty, sprzedawcy i opisie), po typie anomalii oraz po minimalnym poziomie istotności.
WF-12	Alarm-watcher umożliwia podgląd szczegółów wybranego alertu w oknie podręcznym; wyświetlany alert jest „przypięty”, więc napływające alerty nie zmieniają przeglądanej treści.
WF-13	Każdy odebrany alert jest zapisywany do bazy MongoDB (baza anomaly_detection, kolekcja alerts) wraz ze znacznikiem czasu zapisu stored_at.
WF-14	System udostępnia generator testowych transakcji (tx-simulator / mock_tx_producer) pozwalający symulować ruch normalny oraz wstrzykiwać anomalie.
WF-15	Zapisane alerty można przeglądać w panelu webowym Mongo Express (http://localhost:8082).

Wymagania niefunkcjonalne

ID	Kategoria	Wymaganie
WN-01	Wydajność	Przetwarzanie strumieniowe w Apache Flink — alert generowany jest w czasie zbliżonym do rzeczywistego, bez przetwarzania wsadowego.
WN-02	Skalowalność	Komunikacja przez Apache Kafka pozwala skalować producentów i konsumentów niezależnie; reguły detekcji działają na stanie

		partycjonowanym per karta.
WN-03	Niezawodność	Komponenty są od siebie odseparowane — awaria lub restart alarm-watchera nie zatrzymuje detekcji; nieodczytane alerty pozostają w topicu Kafka.
WN-04	Trwałość danych	Każdy alert jest utrwalany w MongoDB i dostępny po restarcie systemu.
WN-05	Przenośność	Cała infrastruktura (Kafka, Zookeeper, Flink, MongoDB, Mongo Express, Kafka UI) uruchamiana jest jednym poleceniem docker compose up w identycznej konfiguracji na każdej maszynie.
WN-06	Modułowość	Każdy komponent (generator, detektor, dashboard, baza) realizuje odrębną funkcję i może być rozwijany oraz testowany niezależnie.
WN-07	Użyteczność	Dashboard działa w terminalu, obsługiwany jest wyłącznie klawiaturą i nie wymaga środowiska graficznego (możliwa praca przez SSH).
WN-08	Rozszerzalność	Dodanie nowej reguły detekcji sprowadza się do dodania kolejnej funkcji detect... w jobie Flink, bez zmian w pozostałych komponentach.
WN-09	Obserwowalność	Stan systemu można weryfikować w panelach Flink UI, Kafka UI i Mongo Express.

3. Architektura systemu

Architektura systemu jest oparta na kilku współpracujących komponentach. Każdy komponent realizuje odrębną funkcję, dzięki czemu system ma charakter modułowy i można niezależnie testować generowanie danych, detekcję anomalii, monitoring oraz zapis wyników.

Element	Opis / wartość
Apache Kafka	Broker komunikatów. Przechowuje topic transactions z transakcjami oraz topic alerts z alertami.
Apache Flink	Silnik przetwarzania strumieniowego. Uruchamia job detekcji anomalii.
MongoDB	Baza danych przechowująca wykryte alerty.
Mongo Express	Panel webowy do podglądu baz i kolekcji MongoDB.
alarm-watcher	Aplikacja terminalowa czytająca alerty z Kafki, pokazująca dashboard i zapisująca alerty w MongoDB.

mock_tx_producer / tx-simulator	Generator przykładowych transakcji kart płatniczych.
Tx-inspector	Przegląd topików transactions w formie czytelnych logów.

Komunikacja między producentem transakcji, detektorem Flink oraz komponentem alarm-watcher odbywa się asynchronicznie za pomocą Apache Kafka. Dzięki temu każdy element systemu może działać niezależnie, a dane są przekazywane przez odpowiednie topiki.

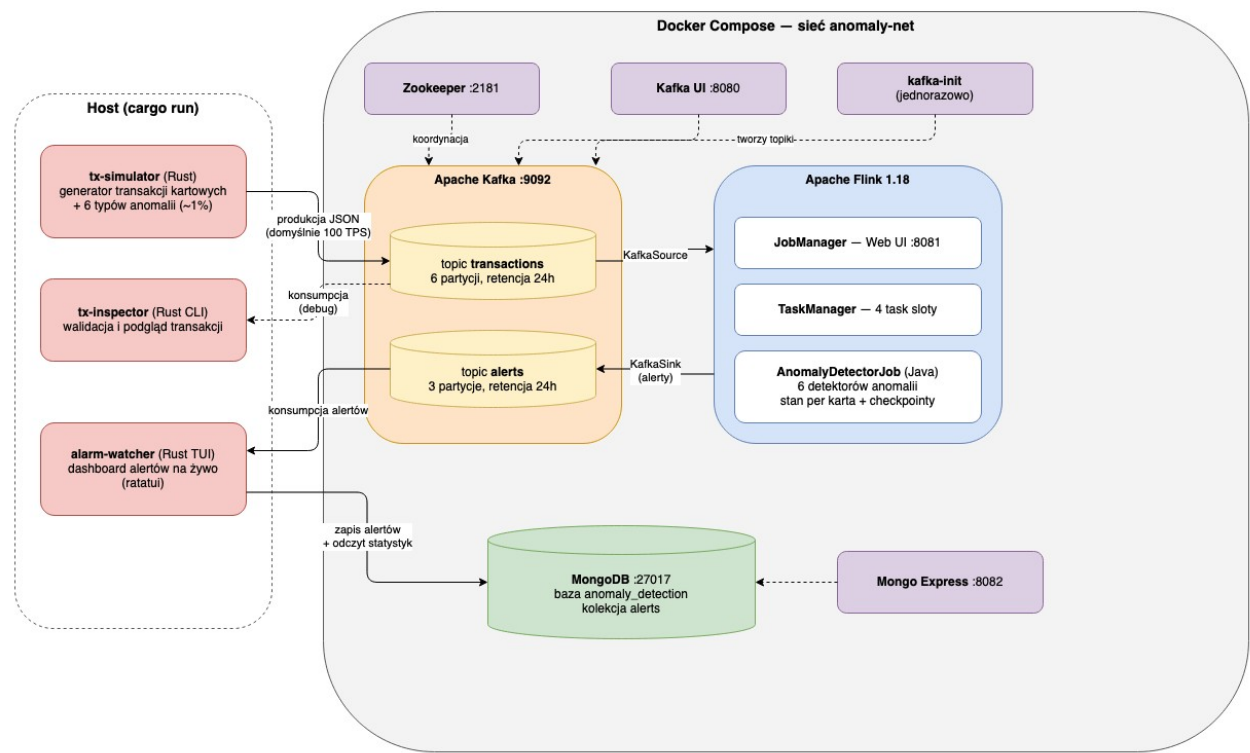


Figure 1: Architektura projektu Card 5 Detector

UI for Apache Kafka

83b5a60 v0.7.2

Topics / alerts

Produce Message

Overview

Messages

Consumers

Settings

Statistics

Partitions

3

Replication Factor

1

URP

0

In Sync Replicas

3 of 3

Type

External

Segment Size

4 MB

Segment Count

3

Clean Up Policy

DELETE

Message Count

6464

Partition ID	Replicas	First Offset	Next Offset	Message Count	
0	1	10401	12542	2141	
1	1	10883	13141	2258	
2	1	10435	12500	2065	

UI for Apache Kafka

83b5a60 v0.7.2

Topics / transactions

Produce Message

Overview

Messages

Consumers

Settings

Statistics

Partitions

6

Replication Factor

1

URP

0

In Sync Replicas

6 of 6

Type

External

Segment Size

17 MB

Segment Count

6

Clean Up Policy

DELETE

Message Count

50144

Partition ID	Replicas	First Offset	Next Offset	Message Count	
0	1	14874	24119	9245	
1	1	16576	26922	10346	
2	1	11262	18305	7043	
3	1	13148	21321	8173	
4	1	12933	21043	8110	
5	1	11667	18894	7227	

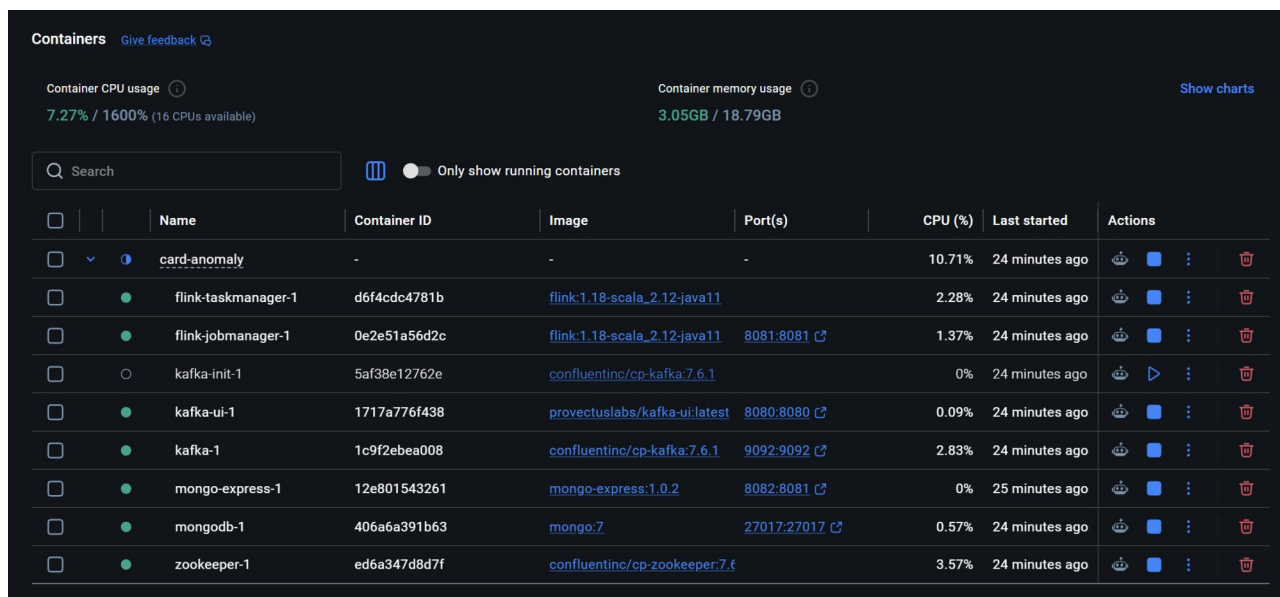
Figure 2 Topiki w Kafce

4. Konfiguracja środowiska

Środowisko projektu zostało uruchomione lokalnie z wykorzystaniem Docker Desktop oraz pliku docker-compose.yaml. Zastosowanie konteneryzacji pozwoliło uruchomić wszystkie wymagane usługi w jednakowej konfiguracji, bez konieczności ręcznej instalacji poszczególnych komponentów systemu.

Po uruchomieniu środowiska sprawdzono, czy wszystkie wymagane kontenery działają poprawnie. Weryfikacja obejmowała kontenery odpowiedzialne za obsługę Apache Kafka, Zookeepera, Apache Flink, MongoDB, Mongo Express oraz Kafka UI. Poprawne uruchomienie tych usług było warunkiem dalszego testowania przepływu danych i działania detektora anomalii.

W ramach konfiguracji przygotowano również plik JAR aplikacji Flink detector, który został zamontowany w kontenerze Flink JobManager i uruchomiony jako job strumieniowy. Dzięki temu detektor mógł działać wewnątrz środowiska Docker i komunikować się z brokerem Kafka po wewnętrznej sieci kontenerów.



The screenshot shows the Docker Desktop interface with the 'Containers' tab selected. It displays a list of running containers with their names, IDs, images, ports, CPU usage, and last started time. The containers are: card-anomaly, flink-taskmanager-1, flink-jobmanager-1, kafka-init-1, kafka-ui-1, kafka-1, mongo-express-1, mongodb-1, and zookeeper-1. The interface also shows overall system usage: 7.27% / 1600% CPU and 3.05GB / 18.79GB memory.

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	card-anomaly	-	-	-	10.71%	24 minutes ago	
☐	flink-taskmanager-1	d6f4cdc4781b	flink:1.18-scala_2.12-java11		2.28%	24 minutes ago	
☐	flink-jobmanager-1	0e2e51a56d2c	flink:1.18-scala_2.12-java11	8081:8081	1.37%	24 minutes ago	
☐	kafka-init-1	5af38e12762e	confluentinc/cp-kafka:7.6.1		0%	24 minutes ago	
☐	kafka-ui-1	1717a776f438	provectuslabs/kafka-ui:latest	8080:8080	0.09%	24 minutes ago	
☐	kafka-1	1c9f2ebea008	confluentinc/cp-kafka:7.6.1	9092:9092	2.83%	24 minutes ago	
☐	mongo-express-1	12e801543261	mongo-express:1.0.2	8082:8081	0%	25 minutes ago	
☐	mongodb-1	406a6a391b63	mongo:7	27017:27017	0.57%	24 minutes ago	
☐	zookeeper-1	ed6a347d8d7f	confluentinc/cp-zookeeper:7.6.1		3.57%	24 minutes ago	

Figure 3 Uruchomione kontenery w Docker Desktop

5. Struktura projektu

Projekt składa się z kilku modułów. Część modułów napisana jest w Rust, natomiast detektor Flink został przygotowany jako osobny moduł Java/Maven.

Element	Opis / wartość
shared	Wspólne struktury danych: Transaction, Alert, AnomalyKind oraz stałe nazw topiców.
tx-simulator	Generator transakcji i anomalii testowych.
tx-inspector	Komponent pomocniczy do podglądu transakcji.
alarm-watcher	Dashboard alertów oraz zapis do MongoDB.
flink-detector	Moduł Java zawierający job Apache Flink odpowiedzialny za wykrywanie anomalii.
docker-compose.yaml	Definicja usług infrastrukturalnych uruchamianych w Dockerze.

6. Implementacja detektora Flink

Najważniejszym elementem części detekcyjnej jest plik AnomalyDetectorJob.java. Definiuje on źródło danych Kafka, funkcję detekcji oraz wyjście Kafka, do którego wysyłane są alerty.

Element	Opis / wartość
pom.xml	Konfiguracja budowania projektu Java/Maven oraz zależności Flinka i Kafka Connectora.
AnomalyDetectorJob.java	Główna klasa joba Flink. Zawiera pipeline oraz reguły detekcji.
Transaction.java	Model transakcji wejściowej odczytywanej z topicu transactions.
Alert.java	Model alertu wysyłanego do topicu alerts.
AnomalyKind.java	Lista obsługiwanych typów anomalii.
GpsCoords.java	Klasa współrzędnych GPS z funkcją liczenia odległości.
CardState.java	Stan historyczny karty używany przez reguły zależne od wcześniejszych transakcji.

Schemat pipeline'u Flinka

```
env.fromSource(source, WatermarkStrategy.noWatermarks(), "Kafka transactions source")
    .flatMap(new DetectorFunction())
    .sinkTo(sink);

env.execute("Card anomaly detector");
```

Konfiguracja KafkaSource

```
KafkaSource<String> source = KafkaSource.<String>builder()
    .setBootstrapServers(broker)
    .setTopics(TOPIC_TRANSACTIONS)
    .setGroupId("flink-detector")
    .setStartingOffsets(OffsetsInitializer.latest())
    .setValueOnlyDeserializer(new SimpleStringSchema())
    .build();
```

Konfiguracja KafkaSink

```
KafkaSink<String> sink = KafkaSink.<String>builder()
    .setBootstrapServers(broker)
    .setRecordSerializer(
        KafkaRecordSerializationSchema.builder()
            .setTopic(TOPIC_ALERTS)
            .setValueSerializationSchema(new SimpleStringSchema())
            .build()
    )
    .build();
```

7. Reguły detekcji anomalii

Detektor analizuje każdą przychodzącą transakcję i uruchamia zestaw reguł. Każda reguła może wygenerować osobny alert. Dzięki temu jedna transakcja może zostać zaklasyfikowana jako więcej niż jeden typ anomalii, jeżeli spełnia kilka warunków jednocześnie.

Element	Opis / wartość
large_amount	Wykrywa transakcję znacząco większą niż wcześniejsze typowe kwoty dla danej karty. Wykorzystuje średnią, odchylenie standardowe i z-score.
high_frequency	Wykrywa dużą liczbę transakcji jednej karty w krótkim oknie czasowym.
limit_exhaustion	Wykrywa sytuację, w której pojedyncza transakcja zużywa co najmniej 95% dostępnego limitu karty.
structuring	Wykrywa kwoty tuż poniżej progów kontrolnych, np. 500, 1000 lub 5000 PLN.
impossible_travel	Wykrywa użycie tej samej karty w dwóch odległych lokalizacjach w czasie wymagającym nierealnej prędkości podróży.

Kolejność uruchamiania reguł

```
detectLargeAmount(tx, state, out);
detectHighFrequency(tx, state, txTimeMillis, out);
detectLimitExhaustion(tx, out);
detectStructuring(tx, out);
detectImpossibleTravel(tx, state, txTimeMillis, out);
```

8. Alarm-watcher i zapis do MongoDB

Komponent alarm-watcher działa jako konsument topicu alerts. Po odebraniu alertu parsuje wiadomość JSON do struktury Alert, prezentuje alert w terminalowym dashboardzie i zapisuje dokument do kolekcji MongoDB.

Dane zapisywane są do bazy anomaly_detection i kolekcji alerts. MongoDB tworzy bazę i kolekcję automatycznie przy pierwszym zapisie dokumentu. Do podglądu wykorzystano Mongo Express dostępny pod adresem <http://localhost:8082>.

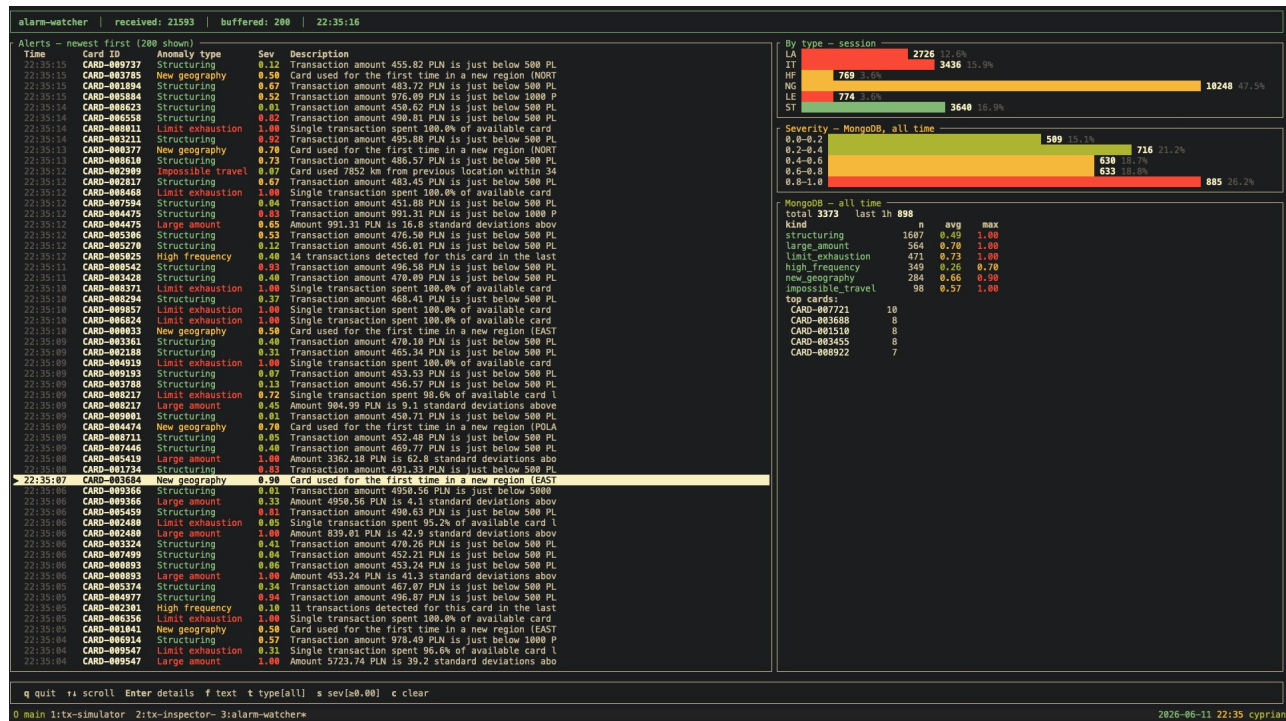


Figure 4 Alarm watcher

Mongo Express

Database: anomaly_detection

Viewing Database: anomaly_detection

Collections

Collection Name

+ Create collection

View

Export

[JSON]

Import

alerts

Del

View

Export

[JSON]

Import

alerts\

Del

Database Stats

Collections (incl. system.namespaces)	2
Data Size	4.85 MB
Storage Size	3.01 MB
Avg Obj Size #	671 Bytes
Objects #	7221
Indexes #	2
Index Size	262 KB

Figure 5 MongoDB

Zapis alertu do MongoDB w alarm-watcher

```
match serde_json::from_str::<serde_json::Value>(&payload) {
  Ok(value) => {
    match mongodb::bson::to_document(&value) {
      Ok(mut document) => {
        document.insert("stored_at", mongodb::bson::DateTime::now());

        if let Err(e) = alerts_collection.insert_one(document).await {
          eprintln!("MongoDB insert error: {e}");
        }
      }
      Err(e) => {
        eprintln!("BSON conversion error: {e}");
      }
    }
  }
  Err(e) => {
    eprintln!("JSON parse for MongoDB error: {e}");
  }
}
```

Obsługa programu alarm-watcher

Alarm-watcher uruchamia się z katalogu card-anomaly poleceniem `cargo run --release -p alarm-watcher`. Domyślnie łączy się z brokerem Kafka pod adresem `localhost:9092` oraz z MongoDB pod adresem `mongodb://admin:secret@localhost:27017` (parametry można zmienić flagami `--broker` i `--mongo-uri`). Aplikacja przejmie cały ekran terminala i na bieżąco wyświetla tabelę alertów (najnowsze na górze) wraz ze statystykami zapisu do MongoDB. Dashboard obsługiwany jest wyłącznie klawiaturą:

Klawisz	Działanie
↓ / j	Przejdź do następnego alertu na liście.
↑ / k	Przejdź do poprzedniego alertu na liście.
Enter	Otwarcie okna ze szczegółami zaznaczonego alertu (ponowne Enter lub Esc zamyka okno).
Esc	Zamknięcie okna szczegółów.
f	Wejście w tryb filtra tekstowego.
t	Cykliczne przełączanie filtra typu anomalii (kolejne typy, na końcu brak filtra).
s	Cykliczne przełączanie progu minimalnej istotności: 0.00 → 0.25 → 0.50 → 0.75 → 0.00.
c	Wyczyszczenie wszystkich filtrów.
q / Ctrl+C	Zakończenie pracy programu.

Po naciśnięciu klawisza `f` aplikacja przechodzi w tryb wpisywania filtra tekstowego: wpisywane znaki budują wzorec wyszukiwania (Backspace usuwa ostatni znak), Enter zatwierdza filtr, a Esc anuluje wpisywanie bez zmiany filtra. Po zatwierdzeniu lista pokazuje tylko alerty, których identyfikator karty, nazwa sprzedawcy lub

opis zawierają wpisany tekst (bez rozróżniania wielkości liter). Filtry tekstowy, typu anomalii i poziomu istotności można łączyć — lista pokazuje alerty spełniające wszystkie warunki jednocześnie; po każdej zmianie filtra zaznaczenie wraca na początek listy.

Okno szczegółów (Enter) pokazuje pełne dane alertu wraz z transakcją, która go wywołała. Treść okna jest zamrożona w momencie otwarcia — napływające w tle alerty nie zmieniają przeglądanego zdarzenia, trafiają jedynie na listę pod spodem.

9. Uruchamianie i testowanie systemu

Po przygotowaniu środowiska uruchomiono job Apache Flink odpowiedzialny za detekcję anomalii. Job został uruchomiony na podstawie pliku JAR wygenerowanego z modułu flink-detector. Poprawność uruchomienia zweryfikowano w panelu Flink UI, gdzie aplikacja była widoczna jako działający job strumieniowy.

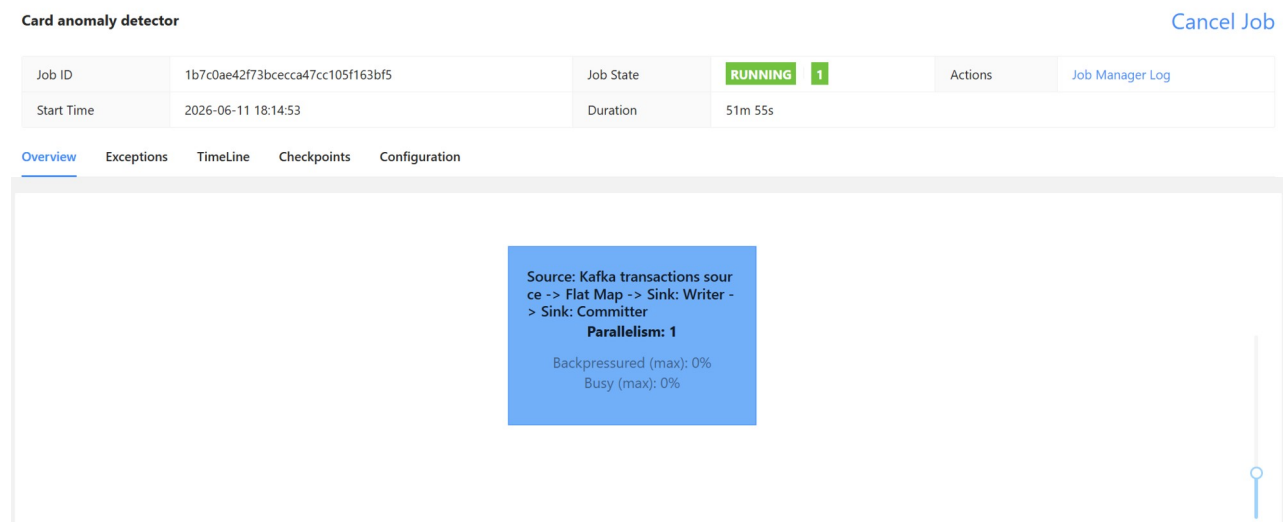


Figure 6 Uruchomiony job Card anomaly detector w panelu Apache Flink

Następnie uruchomiono producenta transakcji oraz komponent alarm-watcher. Producent generował przykładowe transakcje i publikował je do tematu transactions, natomiast alarm-watcher był uruchomiony równolegle w celu obserwacji alertów oraz zapisu wyników do MongoDB.

Test polegał na sprawdzeniu, czy po uruchomieniu producenta pojawiają się nowe wiadomości wejściowe, czy job Flink generuje alerty oraz czy alerty są widoczne w dalszych komponentach systemu.

10. Wyniki działania

Wyniki przedstawiono w poniższej tabeli

Element	Opis / wartość
Kafka transactions	Potwierdzono pojawianie się transakcji generowanych przez producenta.
Flink UI	Potwierdzono działanie joba Card anomaly detector.
Kafka alerts	Potwierdzono pojawianie się alertów w formacie JSON.
alarm-watcher	Potwierdzono podgląd alertów w dashboardzie terminalowym.
MongoDB	Potwierdzono zapis alertów w kolekcji anomaly_detection.alerts.

card_id	user_id	timestamp	anomaly_kind	description	severity	transaction	stored_at
CARD-000130	USER-000030	2026-06-10T16:26:07.780737Z	large_amount	Amount 1757.72 PLN is 49.6 standard deviations ab...	1	<pre>{ "transaction_id": "da79202e-e40c-46df-910d-d6deb852255f", "card_id": "CARD-000130", "user_id": "USER-000030", "timestamp": "2026-06-10T16:26:07.778065+00:00", "location": [2 items], "amount_pln": 1757.72, "remaining_limit_pln": 251.39, "merchant": "Lidl", "injected_anomaly": "large_amount" }</pre>	Wed Jun 10 2026 17:16:10 GMT+0000 (Coordinated Universal Time)

Figure 7 Przykładowy alert przechowywany w MongoDB

11. Wnioski

Projekt pokazuje praktyczne zastosowanie architektury strumieniowej do wykrywania anomalii w transakcjach kart płatniczych. Kafka zapewnia niezależną komunikację między komponentami, Flink umożliwia analizę danych w czasie rzeczywistym, a MongoDB pozwala przechowywać historię alertów do późniejszej analizy.

Najważniejszym rezultatem projektu jest działający przepływ danych od producenta transakcji do bazy danych z alertami. System można rozszerzać o dodatkowe reguły detekcji, dokładniejsze modele statystyczne lub mechanizmy ograniczania liczby powtarzających się alertów.